Lista de exercícios 7

1. Dada uma fila dinâmica **F**, implemente uma função para inverter a ordem dos elementos **F**. Para inverter a ordem dos elementos, faça uso de uma pilha dinâmica **P**.

   O menu deve conter as seguintes opções:

   - Incluir na fila
   - Remover da fila
   - Mostrar fila
   - Inverter fila

2. Considerando duas filas dinâmicas, implemente uma função para concatenar duas filas, as quais devem ser recebidas por parâmetro. Exemplos de declaração da função:

```
Fila concatenaF (Fila  *f1, Fila  *f2);
```

```
void concatenaF (Fila  *f1, Fila  *f2, Fila *fConcatenada);
```

3. Implemente uma função para dividir uma fila dinâmica de valores inteiros. A função deve receber como parâmetro uma fila **F1** com os valores inteiros, uma fila vazia **F2** e um valor inteiro **N.** A função deve dividir a fila **F1** <u>em duas</u>, de tal forma que **F2** comece no primeiro nó (elemento) logo após a primeira ocorrência de **N** na fila original.

   a) Exemplo de declaração da função:

```
void separaF (int N, Fila *F1, Fila *F2);
```

   a) Exemplo de separação:

```
F1[1, 2, 3, 4, 5, 6, 7, 8, 9]
F2[]
separaF (5, &F1, &F2);
F1[1, 2, 3, 4, 5]
F2[6, 7, 8, 9]
```

4. Desenvolva uma função em C++ para combinar elementos em uma fila circular. A fila deve armazenar valores da estrutura Produto apresentada abaixo, sendo que cada Produto possui um nome e uma quantidade. A função deve combinar elementos cujo produto tenha o mesmo nome, somando suas quantidades em um único elemento, removendo elementos redundantes e preservando a ordem do primeiro elemento encontrado (do início para o final da fila).

Estrutura Produto:

```
struct Produto
{
    string nome;
    int quantidade;
};
```

O protótipo da função deve ser:

```
void combinarF(Fila *f);
```

O programa deve exibir na tela:

- Imprimir a fila antes da chamada da função `combinarF`.
- Imprimir o tamanho e o total de elementos da fila.
  Imprimir a fila depois da chamada da função `combinarF`.

Observações:

- Utilizar a estrutura de dados Fila Circular na implementação, com capacidade para armazenar **trinta elementos.**
- É permitido o uso de filas auxiliares.
- Não é permitido o uso da estrutura de dados Pilha.
- A função deve operar sobre a fila recebida por parâmetro, que pode conter diferentes elementos resultantes de operações de enfileiramento e desenfileiramento.

Exemplos de entrada e saída:

- Um exemplo de entrada e saída são apresentados a seguir. O programa desenvolvido deve ser capaz de produzir a saída no formato mostrado.

```
struct Produto{
    string nome;
    int quantidade;
    Produto(string n, int q){
        nome=n;
        quantidade=q;
```

```
    }
};
int main(void){
    Fila f;
    const int tamFila=30;
    inicializaF(&f, tamFila);
    enfileiraF(&f, Produto("melao", 10));
    enfileiraF(&f, Produto("melancia", 5));
    enfileiraF(&f, Produto("abacaxi", 6));
    enfileiraF(&f, Produto("melancia", 3));
    enfileiraF(&f, Produto("melao", 2));
    desenfileiraF(&f);
    desenfileiraF(&f);
    desenfileiraF(&f);
    enfileiraF(&f, Produto("melao", 10));
    enfileiraF(&f, Produto("melancia", 5));
    enfileiraF(&f, Produto("abacaxi", 6));



    cout << "\nFila: {tamanho = " << f.tam << ", total = " << f.total << "}\n";
    mostraF(&f);

    combinarF(&f);

    cout << "\nFila: {tamanho = " << f.tam << ", total = " << f.total << "}\n";
    mostraF(&f);

    destroiF(&f);
    return EXIT_SUCCESS;
}
```

```
Fila: {tamanho = 30, total = 5}
{[melancia, 3], [melao, 2], [melao, 10], [melancia, 5], [abacaxi, 6]}

Fila: {tamanho = 30, total = 3}
{[melancia, 8], [melao, 12], [abacaxi, 6]}
```

5. Crie uma função para ordenar uma fila dinâmica de números inteiros.
    a) Considere um fila com diferentes valores armazenados, fora da ordem;
    b) O uso de vetores ou estruturas auxiliares não é permitido;
    c) Para a ordenação, compare os valores dos dados entre dois nós adjacentes e realize a troca dos valores dos nós (atributo dado), não precisa alterar os endereços de memória;
    d) Dica: uma fila está ordenada, quando não há mais trocas a serem realizadas;
    e) Protótipo da função:

```
void ordenaF(Fila *f, int ordem); //crescente -> ordem=0; decrescente -> ordem=1
```

6. Crie uma função chamada *furaFila*, em que um novo nó deve ser inserido no início fila.
    a) O uso de vetores ou estruturas auxiliares não é permitido;
    b) Realize a atualizações dos ponteiros de forma que um novo criado seja inserido no início da fila;
    c) A função deve manter a ordem relativa dos elementos;
    d) Imprimir a fila antes e depois da chamada da função "*furaFila*";
    e) Protótipo da função:

```
void furaFila(Fila *f, int dado);
```

7. Dada uma fila dinâmica **F1** contendo *n* elementos, escreva um algoritmo que divida os elementos dessa fila em duas novas filas, **F2** e **F3**, de forma intercalada.

Exemplo 1:

```
f1: inicio [1, 20, 3, 40, 5, 60, 7, 80] fim
f2: inicio [1, 3, 5, 7] fim
f3: inicio [20, 40, 60, 80] fim
```

Exemplo 2:

```
f1: inicio [1, 20, 3, 40, 5, 60, 7, 80, 100] fim
f2: inicio [1, 3, 5, 7, 100] fim
f3: inicio [20, 40, 60, 80] fim
```

Observação: caso **F1** possua um número ímpar de elementos, a fila **F2** ficará com um elemento a mais.

O protótipo da função:

```
void separaIntercalado(Fila *f1, Fila *f2, Fila *f3);
```

O programa deve:

- Imprimir a fila **F1** antes da chamada da função `separaIntercalado`.
- Imprimir as filas **F2 e F3** depois da chamada da função `separaIntercalado`.
- Dentro da função `separaIntercalado`, destruir (resetar) previamente as filas **F2** e **F3** antes de inserir os novos elementos.

8. A Veterinária Passo Fundo realiza atendimentos para diferentes tipos de animais. Quando um cliente chega com seu animal na veterinária, ele deve informar os seguintes dados para a criação de uma ficha de atendimento:
    - Ficha de atendimento: nome, idade, tipo (S-selvagem, D-doméstico) e quadro clínico (E-estável, G-grave).

Após o cliente informar os dados ao funcionário da empresa, a ficha de atendimento é criada e, com base nos dados fornecidos, a ficha de atendimento é adicionada em uma das seguintes filas:

- Fila Prioritária – animais com quadro clínico grave;
- Fila Não Prioritária – animais com quadro clínico estável.

O atendimento das filas deve seguir a ordem especificada abaixo:

1. Primeiramente, as fichas da *Fila Prioritária* devem ser atendidas com base nas seguintes regras.
    a. Quando há fichas em ambas as filas, no máximo três fichas consecutivas da *Fila Prioritária* podem ser atendidas, ou seja, a cada três fichas atendidas da *Fila Prioritária,* uma ficha da *Fila Não Prioritária* deve ser atendida (caso houver);
    b. Não havendo fichas na *Fila Não Prioritária,* o sistema pode atender mais que três fichas consecutivas da *Fila Prioritária.*
2. Por fim, quando não há fichas na *Fila Prioritária*, uma ficha da *Fila Não Prioritária* deve ser atendida.

Quando uma ficha é atendida, o funcionário da empresa retira a ficha da fila e guarda em uma pilha, sendo que as fichas dos animais do tipo S são empilhadas na pilha ANIMALS e as fichas dos animais do tipo D são empilhadas na pilha ANIMALD.

Agora você deve escrever um programa que implementa o sistema para controlar tudo que foi descrito:

a) Criar ficha de atendimento e adicionar na fila - o usuário deve informar os dados da ficha e, em seguida, a ficha de atendimento deve ser incluída na fila correspondente;
b) Atender ficha da fila – atender **uma ficha**, respeitando a ordem de atendimento;
c) Mostrar os dados das fichas de atendimento das filas (todas as filas);
d) Listar as informações sobre as filas de forma unificada: total de fichas, a porcentagem de animais com quadro clínico grave em relação ao total e o total de animais domésticos;
e) Listar as fichas já atendidas (as fichas armazenadas nas pilhas);
f) Remover uma ficha da pilha – O usuário deve informar a pilha e, em seguida, o sistema remove a ficha do topo da pilha especificada;
g) Mostrar o nome do animal armazenado nas pilhas que possui a menor idade com o quadro clínico grave.

Então mãos à obra! Analise o problema com atenção e veja quais as funcionalidades que devem ser implementadas.

Dica: A pilha e fila usam o mesmo tipo de dado, ou seja, tipo Ficha.

9. O Despachante Passo Fundo é especializado em requerimentos e encaminhamentos na documentação de veículos. Quando um cliente chega na empresa, ele deve informar os seguintes dados para a criação de uma ficha de atendimento: nome, idade, sexo, cidade e placa do veículo.

Após o preenchido dos dados da ficha, o funcionário guarda a ficha em uma pilha, sendo que a ficha cuja a cidade é Passo Fundo são empilhadas em PILHAPF e as fichas de outras cidades são empilhadas na pilha PILHAGERAL.

Agora você deve escrever um programa que implementa o sistema para controlar tudo que foi descrito:

h) Criar ficha de atendimento - o usuário deve informar os dados da ficha de atendimento e, em seguida, a ficha deve ser empilhada na pilha correspondente;

i) Mostrar os dados das fichas de atendimento de todos os clientes (das duas as pilhas).

j) Listar as fichas da pilha PILHAPF na ordem em que eles foram preenchidas (imprimir a pilha na ordem inversa);

k) Mostrar informações dos clientes da pilha PILHAGERAL: números de clientes de cada sexo e média aritmética da idade dos clientes.

l) Mostrar a placa do veículo da pessoa mais velha (os dados estão armazenados nas pilhas PILHAPF e PILHAGERAL).